

Lab Sheet 4 — UDP Socket Programming

Connectionless Chat Server and Broadcast Client

Course	977-337 Network Programming and Simulation
Student	Hadis Rodpradit
Student ID	6630613039
Program	Digital Engineering (International), College of Computing, PSU Phuket
Environment	Ubuntu 24.04.4 LTS (noble) · Python 3.12.3 · venv: netprog-env

This report documents completion of the six lab tasks for Lab Sheet 4 (a connectionless UDP chat server that tracks client addresses and broadcasts every message to all registered clients, together with its client program), followed by the verification checklist and reflection answers. Each task is evidenced by terminal screenshots.

Task 1 — Single client UDP echo (Part D)

```
diz@diz: ~/netprog-2026/lab04_udp_chat
diz@diz:~/netprog-2026/lab04_udp_chat$
diz@diz:~/netprog-2026/lab04_udp_chat$
diz@diz:~/netprog-2026/lab04_udp_chat$ python3 udp_server.py
UDP chat server listening on 0.0.0.0:6000
New client: ('127.0.0.1', 37977)
```

Server side

```
diz@diz: ~/netprog-2026/lab04_udp_chat
diz@diz:~/netprog-2026/lab04_udp_chat$ python3 udp_client.py
UDP chat client. Type a message and press Enter ('quit' to exit).
> hello
>
[127.0.0.1:37977]: hello
```

Client side

Figure 1 — The server binds 0.0.0.0:6000 and registers the first datagram it receives as a new client (127.0.0.1:37977); the client sends “hello” and receives the same message back, tagged with the sender’s address. No connection is established at any point.

Task 2 — Broadcast to three clients (Part E)

```
diz@diz: ~/netprog-2026/lab04_udp_chat x diz@diz: ~/netprog-2026/lab04_udp_chat
diz@diz:~/netprog-2026/lab04_udp_chat$ python3 udp_server.py
UDP chat server listening on 0.0.0.0:6000
New client: ('127.0.0.1', 42877)
New client: ('127.0.0.1', 44748)
New client: ('127.0.0.1', 53053)
```

Server — three client addresses registered

```
diz@diz: ~/netprog-2026/lab04_udp_chat x diz@diz: ~/netprog-2026/lab04_udp_chat
diz@diz:~/netprog-2026/lab04_udp_chat$ python3 udp_client.py
UDP chat client. Type a message and press Enter ('quit' to exit).
> hello
[127.0.0.1:42877]: hello
> hi
[127.0.0.1:42877]: hi
>
[127.0.0.1:44748]: hi
>
[127.0.0.1:53053]: hi
>
```

Client 1 — port 42877

```
diz@diz: ~/netprog-2026/lab04_udp_chat x diz@diz: ~/netprog-2026/lab04_udp_chat
diz@diz:~/netprog-2026/lab04_udp_chat$ python3 udp_client.py
UDP chat client. Type a message and press Enter ('quit' to exit).
> hi
[127.0.0.1:44748]: hi
>
[127.0.0.1:53053]: hi
>
```

Client 2 — port 44748

```
diz@diz: ~/netprog-2026/lab04_udp_chat x diz@diz: ~/netprog-2026/lab04_udp_chat
diz@diz:~/netprog-2026/lab04_udp_chat$ python3 udp_client.py
UDP chat client. Type a message and press Enter ('quit' to exit).
> hi
[127.0.0.1:53053]: hi
>
```

Client 3 — port 53053

Figure 2 — The server keeps a set of client addresses and re-sends every incoming datagram to all of them through a single socket, so each message reaches all three clients including the sender. Client 1 joined first and therefore sees all three messages, while clients 2 and 3 see only the traffic sent after they registered. There is no `accept()` and no per-client thread — each client runs one daemon listener thread so that incoming broadcasts still print while it is blocked on `input()`.

Task 3 — Sending after the server is stopped: UDP vs TCP (Part F)

```
diz@diz: ~/netprog-2026/lab04_udp_chat  ×  diz@diz: ~/netprog-2026/

diz@diz:~/netprog-2026/lab04_udp_chat$ python3 udp_client.py
UDP chat client. Type a message and press Enter ('quit' to exit).
> hi
>
[127.0.0.1:53053]: hi
> test after server down
> █
```

UDP client — no error, no reply

```
diz@diz: ~/netprog-2026/lab04_u...  ×  diz@diz: ~/netprog-2026/lab04_u...  ×  diz@diz: ~/netprog-2026/

diz@diz:~/netprog-2026/lab03_tcp_echo$ python3 client.py
Connected to server. Type a message and press Enter ('quit' to exit).
> hello
Echoed back: HELLO
> test after server down
Echoed back:
>
Traceback (most recent call last):
  File "/home/diz/netprog-2026/lab03_tcp_echo/client.py", line 28, in <module>
    main()
  File "/home/diz/netprog-2026/lab03_tcp_echo/client.py", line 20, in main
    client_sock.sendall(message.encode("utf-8"))
BrokenPipeError: [Errno 32] Broken pipe
```

TCP client (Lab 3) — BrokenPipeError

Figure 3 — The same experiment on both protocols: the server is killed, then the client sends one more message. UDP accepts it silently; TCP raises *BrokenPipeError* [Errno 32].

Explanation. On UDP the client keeps running and sending with no error at all. `sendto()` simply hands a datagram to the operating system and returns — there is no connection to break, so the socket has no idea the peer is gone. The datagram is delivered to a port nobody is listening on and is discarded; the client just never receives a reply. On TCP the client fails immediately with `BrokenPipeError` [Errno 32]: the connection was already torn down when the server died, and `sendall()` cannot write into a socket whose connection no longer exists, so the OS raises the error back to the application. This is the practical difference between a connectionless and a connection-oriented protocol — only TCP can tell you that the other side is gone.

Task 4 — Two clients sending at the same time

```
diz@diz: ~/netprog-2026/lab04_udp_chat x diz@diz: ~/netprog-2026/lab04_udp_chat x
diz@diz:~/netprog-2026/lab04_udp_chat$ python3 udp_client.py
UDP chat client. Type a message and press Enter ('quit' to exit).
> a
>
[127.0.0.1:60504]: a
>
[127.0.0.1:58173]: b
> AAAA
>
[127.0.0.1:60504]: AAAA
>
[127.0.0.1:58173]: BBBB
>
```

Client A — port 60504, sends AAAA

```
diz@diz: ~/netprog-2026/lab04_udp_chat x diz@diz: ~/netprog-2026/lab04_udp_chat x
diz@diz:~/netprog-2026/lab04_udp_chat$ python3 udp_client.py
UDP chat client. Type a message and press Enter ('quit' to exit).
> b
>
[127.0.0.1:58173]: b
> BBBB
[127.0.0.1:60504]: AAAA
>
>
[127.0.0.1:58173]: BBBB
>
```

Client B — port 58173, sends BBBB

Figure 4 — Both clients send at nearly the same moment. Each client receives both datagrams, and in this run AAAA is displayed before BBBB on both sides.

Explanation. In this test AAAA arrived before BBBB, matching the order in which the two Enter keys were actually pressed, and both clients displayed the messages in the same order. That happened because everything ran over the loopback interface, where there is effectively no queueing, no routing and no competing traffic, so datagrams arrive in the order they were sent almost every time. It is not a guarantee. UDP does not preserve ordering: each datagram is routed independently, and across a real network two messages sent close together can take different paths or hit different queues, so BBBB could well be delivered before AAAA, or not at all. TCP is the opposite — sequence numbers make it an ordered, reliable byte stream, and the receiver never sees data out of order.

Task 5 — Identifying the sender with an [IP:port] prefix

Because `recvfrom()` returns the sender's address alongside the payload, the server can prefix every broadcast message with the originating [IP:port] before re-sending it. This is what makes the chat readable: without the prefix, a client receiving several broadcasts would have no way to tell which participant wrote which line, since a single UDP socket carries traffic from every client. The behaviour is already visible in the screenshots above rather than needing a separate test — in Figure 2 each client shows lines such as `[127.0.0.1:42877]: hi` and `[127.0.0.1:53053]: hi` from the three different senders, and in Figure 4 the two competing messages are clearly attributed as `[127.0.0.1:60504]: AAAA` and `[127.0.0.1:58173]: BBBB`. The client's own messages come back with its own port in the prefix, confirming that the server broadcasts to every registered address including the sender.

Task 6 — Applications where losing packets is acceptable

Video calls, online games and live streaming are the clearest examples, and all three are built on UDP. Their data is only useful while it is current: a late audio or video frame belongs to a moment that has already passed, and a dropped position update in a game is superseded by the next one a few milliseconds later — so re-sending either one would waste bandwidth and push the stream behind real time instead of repairing anything. Skipping the loss is the better outcome; the user hears a brief glitch and everything stays synchronised. UDP is faster here precisely because it does none of that work — no handshake, no acknowledgements, no retransmission timers and no head-of-line blocking while a lost segment is recovered — which is exactly the trade-off real-time media wants.

Verification Checklist

#	Checkpoint	Status
1	udp_server.py starts and prints "UDP echo server listening on 0.0.0.0:6000"	Done
2	A single client can send a message and receive the identical message echoed back (Part D)	Done
3	Starting several clients simultaneously does not hang, unlike Lab 3's unthreaded TCP server	Done
4	Broadcast chat (Part E) delivers each message to all connected clients, including the sender	Done
5	Server-down test (Part F) performed for both UDP and TCP, with the difference clearly captured	Done
6	Sender-address prefix (Task 5) works correctly	Done

Reflection Questions

1. Explain how `recvfrom()` differs from `recv()`, and why UDP's API has no `listen()` or `accept()` at all.

`recvfrom()` returns a tuple of (data, addr): both the payload and the address of whoever sent that particular datagram, so the server learns the sender of every single packet as it arrives. `recv()` returns only the data, because a TCP socket is already bound to one specific peer — the identity of the other side is fixed by the connection, so it does not need to be repeated per message. A UDP server has no `listen()` or `accept()` because there is no connection to set up in the first place: nothing is negotiated, nothing is queued for acceptance, and the server simply binds a port and starts reading whatever datagrams arrive on it.

2. Why can a UDP server “broadcast” to multiple clients just by remembering their addresses, while a TCP server needs a separate persistent connection (and usually a separate thread) per client to do the same thing?

Because a UDP address is all the server needs in order to send: the same single socket can `sendto()` any (IP, port) it likes, so remembering the addresses it has already heard from is literally enough to reach every client from one loop, with no per-client socket and no per-client thread. In TCP the destination is not a parameter of `send()` — it is a property of the connection, so reaching N clients requires N accepted sockets, and because `recv()` blocks, each of those needs its own thread just to stay responsive. The only threading my UDP program needs is a lock around the shared clients set on the server and one background listener thread in the client (so it can receive broadcasts while blocked on `input()`) — neither of which scales with the number of clients. The trade-off is that the server is never told when a client disappears, so its address list can accumulate stale entries.

3. Give an example of a real application where occasional packet loss is acceptable, and explain why UDP's lack of retransmission is actually a reasonable trade-off there rather than a flaw.

Voice and video calling (VoIP / video conferencing) is a good example. Its data is only useful while it is current: by the time a lost audio frame could be requested and resent, the moment it belonged to has already been played, so the retransmitted copy has nothing left to contribute. Dropping it costs a brief glitch, while waiting for it would add latency to everything behind it and push the whole call out of sync — which is far worse for a conversation. So the absence of retransmission is not a missing feature here; it is the correct behaviour for data that expires, and it is why real-time media is built on UDP rather than TCP.